# 空间融合事件驱动加速器 Sparkle 技术手册（S）

## 目录


1 研究动机.....1

2 处理器架构设计.....2

2.1 整体架构.....2

2.2 工作流概览.....3

3 核心电路设计.....4

3.1 空间引擎.....4

3.2 向量引擎.....5

3.3 阈值引擎.....5

4 数据流设计.....5

5 缓存区设计.....6

5.1 膜电位缓存.....6

5.2 权重缓存.....6

5.3 脉冲缓存.....7

6 数据调度器.....11

6.1 膜电位取址和重排.....11

6.2 AER 乱序与流水线冒险.....15

参考文献.....17

# 1 研究动机

根据目前的文献调研情况，SNN 加速器可分为 2 个流派：

1. 密集计算。借鉴 ANN 加速器，使用脉动阵列进行加速，脉冲在特征图上以 0/1 表示。由于脉动阵列需要连续输入数据，因此无法利用脉冲稀疏性，或者只能采用复杂方法压缩数据流；

2. 事件驱动。可以 100%利用脉冲稀疏性，但无法采用脉动阵列等经典加速结构。事件驱动的卷积计算方法如图 1：

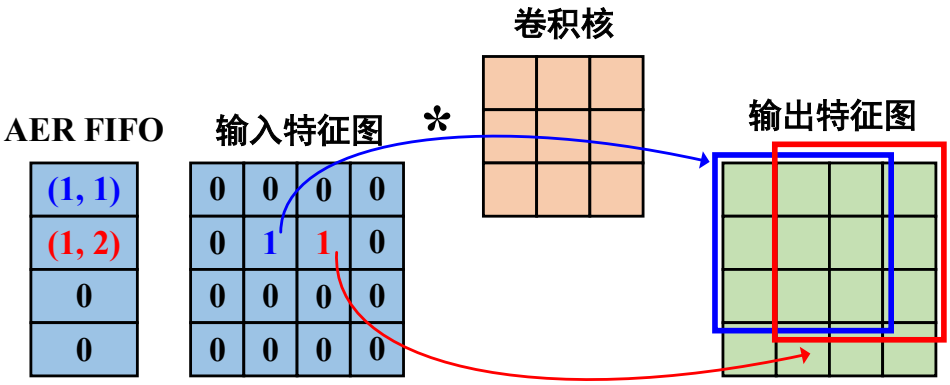
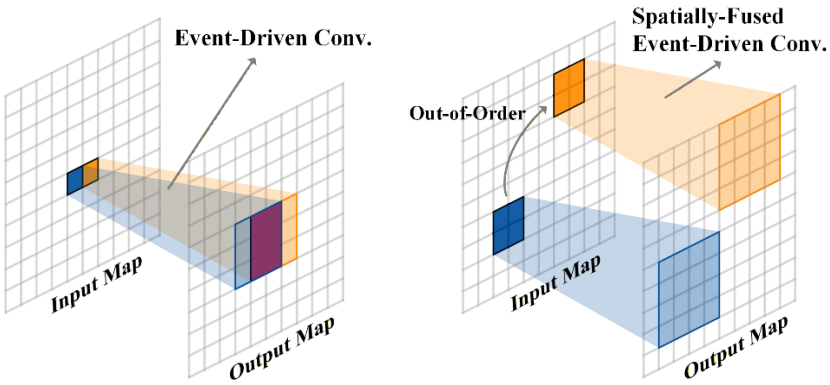



图 1. 事件驱动卷积

以图 1 中的 3×3 卷积为例，每来一个脉冲，直接把卷积核旋转 180°叠加在输出特征图上即可，卷积核复用，用 9 个 PE 并行更新 9 个神经元膜电位。为了提高吞吐率，在硬件上通常采用流水方式处理，即不间断读取脉冲，PE 内部设计成多级流水线。但这种方式存在如下几个瓶颈：

- 重复访问膜电位存储器。如图 1 所示，一旦输入脉冲相邻，输出特征图中将有 6 个膜电位（2/3）要重复访问，即使脉冲之间位置间隔为 1，也有 3 个要重复访问，这带来了额外的能量开销；
- 流水线写后读（RAW）冒险。如果蓝框中的膜电位算完了还没有写入 Mem，或者还没有算完，红框又要读取膜电位，此时读的还是旧值，将会发生错误。为解决这个问题，可以采用旁路转发，或停顿流水线来解决[1]，但这带来了额外的电路开销，或降低流水线吞吐量；
- AER 格式脉冲存储开销。尽管利用 AER 格式存储脉冲可以利用脉冲稀疏性，然而 AER 需要$(c,x,y)$多比特存储，相比于直接存储 0/1，未必能带来存储优势。经初步估算，对于 Spiking VGG11 第一层，脉冲稀疏性要大于 95%在存储开销方面才能取得优势。但一般来说脉冲稀疏性只有 80%-90%，因此存储开销可能会成为瓶颈。

为解决上述问题，本文提出空间融合事件驱动 + 乱序传播机制，如图 2(b)所示，将相邻的脉冲打包为一个事件一起处理，同时打乱处理顺序，尽可能避免处理相邻的事件。如图 2(b)所示，2×2 邻域的脉冲被打包为一个 AER Tile 一起处理，对应输出通道 4×4 膜电位更新。




(a) (b)

图 2. (a)事件驱动卷积；(b)空间融合事件驱动卷积

该方案的优势如下：

- 减少膜电位的重复访问。首先，原本 2×2 邻域内脉冲对应的膜电位重复访问被避免；其次，通过乱序传播减少了 Tile 之前相邻的情况；
- 降低流水线冒险几率。因为膜电位重复访问的概率降低，因此流水线因冒险导致的性能损失也会降低；
- 提升了处理速度，就理想情况而言，假设每个 Tile 里的脉冲都是填满的，处理器吞吐率可提升 4 倍；
- 减少了 AER 存储开销。以 Spiking VGG11 为例，其第一层特征图大小为 224×224×64，因此 $x$、$y$、$c$ 分别需要 8b、8b、6b 存储，共需 24b；而若采用 2×2 AER Tile，只需要一个 Tile 地址 $x'$、$y'$、$c$ 和一个 4-bit *bitmap* 指示哪些位置有脉冲即可。由于划分为 2×2 Tile，特征图长宽相当于减半，此时每 4 个脉冲（假设 Tile 填满）只需要 7b+7b+6b+4b=24b 存储。后文还将讲到，通过一些优化技术可以避免存储 6-bit 的通道 $c$。因此，原本 4 个脉冲需要 4×24b=96b 存储，现在仅需要 18b，脉冲的存储空间需求最大可降低(96 - 18) / 96 = 81.25%。

# 2 处理器架构设计

## 2.1 整体架构

图 3 展示了处理器架构。处理器采用了层次化存储，在片外存储器（DDR）中存储编码的数据集、整个网络的权重，以及中间生成的脉冲。片内包含权重缓

存、脉冲缓存和膜电位缓存。核心计算引擎包括空间引擎、向量引擎和阈值引擎。其中空间引擎专门负责计算 3×3 卷积层，向量引擎负责计算全连接层，阈值引擎负责遍历膜电位、加偏移和发射脉冲，同时还负责池化层的计算。考虑到深度 SNN 中卷积层的计算占 90%以上，因此可多个空间引擎并行以提升吞吐率。记空间引擎的数量为 $N_S$，向量引擎和阈值引擎的数量均为 1。数据调度器是局部缓存和计算引擎之间的数据中转站，负责所有的地址计算和数据传输。

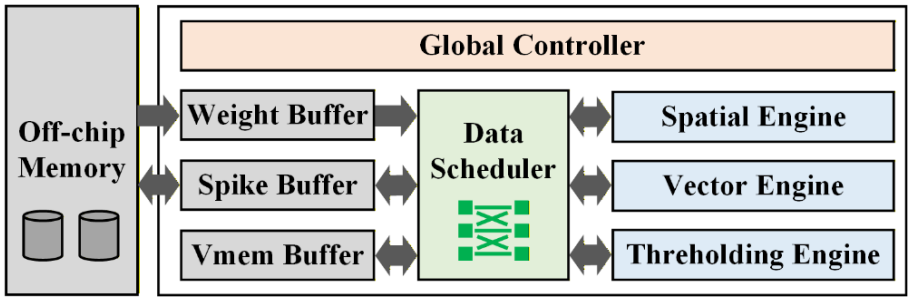



图 3. 处理器架构

## 2.2 工作流概览

图 4 简要展示了处理器的工作流，为方便说明此处假设 $N_S$=1。处理器逐层（$L$）处理，在输出通道（$C_{out}$）循环下，如果当前层是 3×3 卷积层，启动空间引擎进行计算；如果是全连接层，则启动向量引擎进行计算。时间步循环（$T$）被放置在输出通道循环内，对于每个时间步，依次遍历所有输入通道（$C_{in}$）的 AER（$N_{AER}$）并更新膜电位，当当前时间步所有输入通道 AER 处理完后，启动阈值单元遍历当前输出通道膜电位进行阈值判断和脉冲发射。当处理完所有时间步所有输入通道的 AER 后，一个输出通道处理结束，然后重复上述过程，处理下一个输出通道。

```
for l = 0 to L - 1:
    for oc = 0 to C_out - 1:
        if 3×3 Conv:
            Spatial Engine start;
        else if FC:
            Vector Engine start;
        for t = 0 to T - 1:
            for ic = 0 to C_in - 1:
                for a = 0 to N_AER - 1:
                    Update Vmem;
                end
            end
            Threholding Engine start;
        end
    end
end
```

图 4. 处理器工作流

对比先前 LeNet5 加速器设计文档可以发现，本设计的工作流有如下特点：

- 相比于密集计算范式的 7 层循环，事件驱动范式只有 5 层循环，这正是稀疏计算的优势；
- 时间步循环被放置在内层，意味着每个输出通道都要一口气计算完所有时间步。这样带来的好处是只需在片内缓存一个输出通道的膜电位，由于膜电位一般精度较高，因此大幅减少了存储开销，但代价是每个输出通道都要重复读取输入脉冲。

# 3 核心电路设计

## 3.1 空间引擎

图 5 展示了空间引擎的电路实现，数据调度器把脉冲、权重和旧膜电位发送给空间引擎，然后由空间映射单元将这些数据正确分配给神经元积分阵列计算。如图 2(b)所示，当采用 2×2 AER Tile 时，可并行处理 16 个神经元，但是某些神经元需要同时累加多个突触权重。如图 5 所示，我们将其分为 3 类：1）角点神经元，只需计算 1 个权重+1 个膜电位；2）边缘神经元，需计算 2 个权重+1 个膜电位；3）中心神经元，需计算 4 个权重+1 个膜电位。可以看出神经元积分阵列只需要使用加法器和加法树就可以完成计算。文献[2]的研究表明，每个 DSP48E2 可以同时完成 4 个 8-bit 加法计算，相较于使用 LUT 实现一个 2 级加法树，使用 DSP 更加节约资源，且可以达到更高的时钟频率。因此，本文也采用 DSP 来实现所有的加法计算。

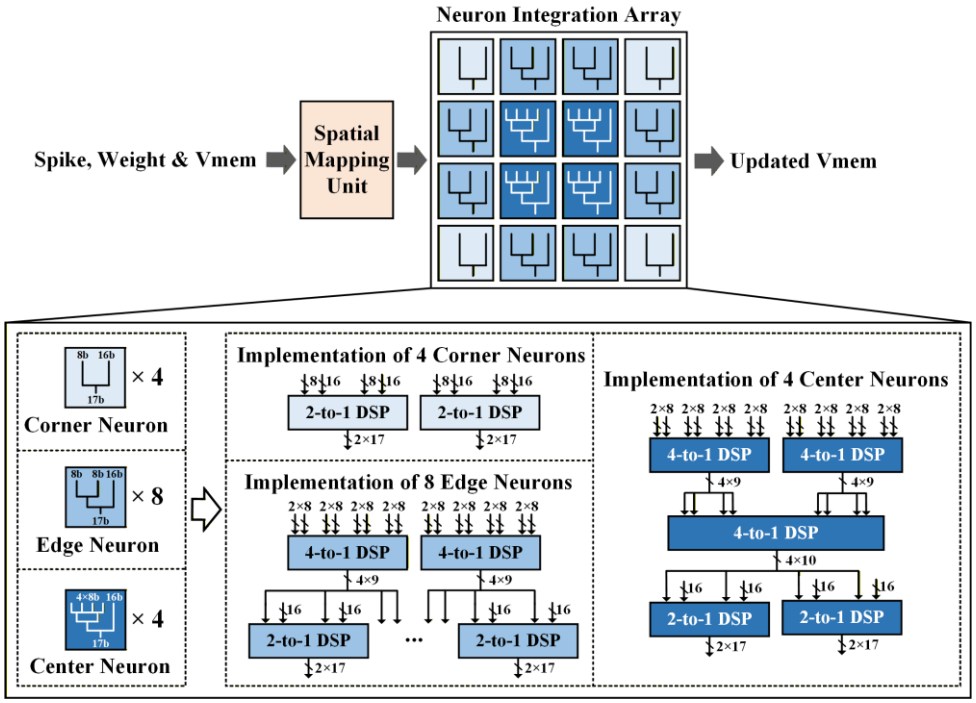



图 5. 空间引擎电路实现

对于 4 个角点神经元，需要使用 2 个 DSP 单元实现，每个被配置成 2-to-1 模式，同时计算两路（8b+16b），最终得到 4 个 17b 膜电位结果；对于 8 个边缘神经元，需要使用 6 个 DSP 级联实现，第一级的 2 个 DSP 被配置为 4-to-1 模式，同时计算四路（8b+8b），得到 8 个 9b 中间结果。第二级的 4 个 DSP 被配置为 2-to-1 模式，同时计算两路（9b+16b），最终得到 8 个 17b 膜电位结果；对于 4 个中心神经元，需要使用 5 个 DSP 级联实现，具体请查看图 5，此处不再赘述。

## 3.2 向量引擎

向量引擎的实现较为简单，如图 6 所示，只需要使用 2-to-1 DSP 实现（8b+16b）加法即可，向量引擎的数据流与空间引擎不同，采用了输出固定（见后文），每次固定计算 16 个神经元的膜电位，因此只需要 8 个 DSP 实现。

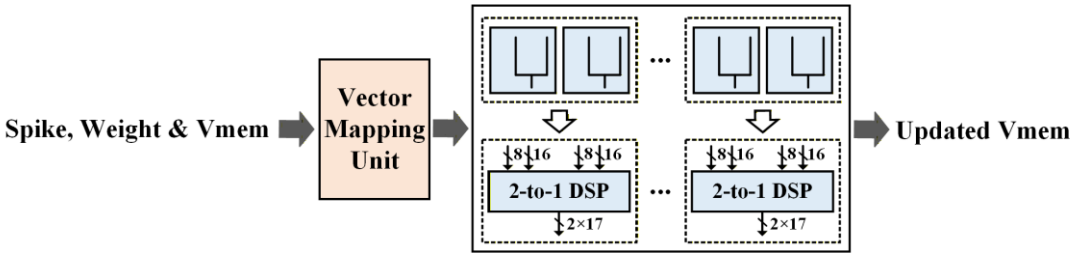



图 6. 向量引擎电路实现

## 3.3 阈值引擎

阈值引擎有如下功能：

- 当前输出通道的每个时间步处理完后，按 4×4 大小扫描该输出通道所有膜电位，加上偏移并与阈值进行比较，最多同时发射 4 个 AER Tile；
- 如果当前卷积层后包含 Pooling 层（默认 2×2 Max Pooling），先将 4×4 的脉冲块看作 4 个 2×2 脉冲块，每个脉冲块通过内部的按位逻辑或操作压缩为 1bit，4 个 2×2 脉冲块压缩后形成 1 个 2×2 脉冲块，地址右移一位形成 AER Tile，最多发射 1 个 AER Tile；
- 对加偏置后的膜电位进行软复位和衰减，若此时为时间窗口中的最后一个时间步，直接对膜电位清零。

阈值引擎是必不可少的，不能把发射脉冲的功能合并到空间引擎或向量引擎中，这是因为最后一个输入通道的脉冲处理完后才能得到最终有效膜电位用于阈值判断，而不能在中途发射脉冲。但是最后一个输入通道的脉冲无法激活所有膜电位，因此必须重新遍历一遍。

# 4 数据流设计

本文采用混合数据流设计，以同时兼容 3×3 卷积和全连接计算：

- 第一种面向 3×3 卷积，类似于权重固定（WS）。在计算当前输出通道时，

是按输入通道顺序处理脉冲，即第一个输入通道处理完接着处理第二个，因此在处理每个输入通道脉冲时，卷积核是固定的。每来一个 AER Tile 就更新 16 个膜电位，直到把所有时间步、所有输入通道脉冲计算完再跳转到下一组输出通道，重复上述过程；

- 第二种面向全连接，类似于输出固定（OS）。对于全连接层而言，每次固定计算 16 个神经元，每来一个输入脉冲 Tile 要把它拆分成单个脉冲依次处理，对于每一个脉冲，都相应读取 16 个突触权重累加到膜电位上，直到把所有时间步、所有输入通道脉冲计算完再跳转到下一组 16 个神经元，重复上述过程。

再简单分析一下两种数据流的区别：

- 对于 3×3 Conv 和 FC，输入脉冲的方式基本是一样的，唯一区别是空间引擎每次处理一个 AER Tile，而向量引擎要拆开逐个处理。在权重方面的区别是，空间引擎在处理每个输入通道时，读 9 个权重且只读一次，而向量引擎处理每个脉冲都要读 16 个不同的权重；在膜电位方面的区别是，空间引擎处理每个 AER Tile 都要读写 16 个不同的膜电位，而向量引擎是固定 16 个膜电位，直到所有输入脉冲处理完才写进存储器。

# 5 缓存区设计

## 5.1 膜电位缓存

目标模型是 Spiking VGG11，在训练 CIFAR10 数据集时，其最大特征图尺寸为 32×32。在 2×2 AER Tile 配置下，由于空间引擎每次并行处理 16 个膜电位，因此需要 16 个 Mem Bank，每个 Bank 的容量为 16b×32×32/16=1Kb，片上一块 BRAM 容量是 36Kb，如果用 BRAM 实现太浪费，直接使用 LUTRAM 实现即可，LUTRAM 在局部布线上更灵活，还有助于提升空间引擎的时钟频率。在配置为双端口的情况下估计每个 Bank 仅需要 32 个 LUT 实现，需为每个空间引擎配置 16×32=512 个 LUT。假设 $N_S$=16，共需要 16×512=8192 个 LUT，xczu28dr 中有 213600 个 LUTRAM，仅占用约 4%。

由于此时膜电位缓存开销非常小，因此很有必要采用乒乓缓存来掩盖膜电位清零开销，当空间引擎在 Ping 缓存中进行计算时，控制器可以将 Pong 缓存清零，以消除通道切换时的延迟。关于深层网络中缓存利用率下降的问题，由于缓存基础容量仅为 1Kb，这种逻辑资源“浪费”在可控范围内，无需再为深层专门引入复杂的动态拼接控制逻辑，保持硬件设计的极简和高频优势更为重要。

## 5.2 权重缓存

首先考虑大小的设计。VGG11 中最大的卷积核尺寸为 512×512×3×3，假设

$N_S$=16，每次计算时前需要缓存 8b×512×16×3×3=576 Kb，共需要 576 / 36 = 16 块 BRAM。针对小模型（S）版本的全连接层 512×10，采用 16 个神经元输出固定的数据流（实际仅需有效计算 10 个输出），因此单次计算只需缓存 8b×512×16 = 64 Kb 的权重，在容量需求上远小于大模型（L）版本。需要注意，卷积层和全连接层有一个重要区别是卷积层每次需要读取 9 个权重，而全连接层需要读取 16 个。综合考虑上述信息，提出一种 Bank 动态拼接方案：将权重缓存划分为 18 个 Bank，每个用一块 BRAM 实现。在计算卷积层时把 2 个 Bank 拼接成一个大 Bank，逻辑上变成 9 个大 Bank，相当于扩展了 Bank 的深度，可以满足卷积层存储和读取需求；而在计算全连接层时，仅使用其中的 16 个 Bank，剩余 2 个 Bank 可以关闭时钟/使能以省电。通过这种动态拼接设计，既满足了深层卷积的容量需求，又满足了全连接层的高带宽需求，而且硬件代价较小，仅需额外增加一些控制逻辑。需要注意的是，与膜电位缓存不同，权重缓存是所有空间引擎共享的，而不是每个空间引擎单独配置一个，因此每次需要花 16 个周期完成权重预加载，由于处理每个输入通道期间权重是固定的，所以这个开销可以接受。假设这里成为瓶颈，可以在空间引擎内采用乒乓权重寄存器，但大概率不会。

接下来考虑延迟问题。由于权重缓存需要和片外 DDR 交互，数据访问延迟是很大的，因此必须要考虑如何掩盖读取延迟。这里采用 PingPong 缓存策略，直接将权重缓存大小翻倍成 18×2 = 36 块 BRAM（1296Kb），当空间/向量引擎在使用 Ping 缓存计算这一组通道/神经元时，将计算下一组计算所需的权重从 DDR 读到 Pong 缓存中，这样只要计算延迟大于读取延迟，就可以通过 PingPong 操作完美掩盖读取延迟。计算延迟取决于脉冲稀疏性，可以做完再来分析。

## 5.3 脉冲缓存

首先来讨论 AER 位宽的问题。在推理 CIFAR10 数据集时，Spiking VGG11 最大层的尺寸降为 32×32，由于采用 2×2 Tile，在硬件视角下看作 16×16，因此每个 AER 的长、宽坐标各仅需要 4-bit 即可表示，Bitmap 需要 4-bit，总共需要 12bit。本文采用和文献[1]一样的方法来避免存储通道信息。首先阈值引擎是按 4×4 大小扫描每个通道膜电位，最多同时发射 4 个 AER Tile，因此需要采用 4 个小 FIFO 缓存后发送至片外，在扫描完最后一个 4×4 块后，发送 4 个 CEnd 数据包（可用全 0 表示）到 4 个 FIFO 中，表示当前通道的脉冲发射已经结束。输入脉冲缓存有四个 Bank，数据调度器逐 Bank 读取脉冲，每次读到 CEnd 时就跳转到下一个 Bank，当读到 4 个 CEnd 说明当前输入通道处理完毕，输入通道计数器+1。通过这种方式，控制模块就可以实时监控输入通道处理进度以进行控制。

位宽和片内脉冲处理方式确定后，接下来考虑脉冲缓存和 DDR 的交互设计，这里有以下两个难点：1）和权重缓存不同，脉冲还需要写入 DDR；2）各层脉冲

数量是动态变化、不可预测的，难以固定缓存多少通道的脉冲，DDR 中也不好直接存储。为了解决这些问题，本文提出一种动态稀疏内存管理机制。首先讨论在 DDR 中如何存储脉冲数据，这里把 DDR 划分为索引区和数据区两个区域，其起始地址分别记为 IdxBaseAddr 和 DataBaseAddr，如图 7 所示：

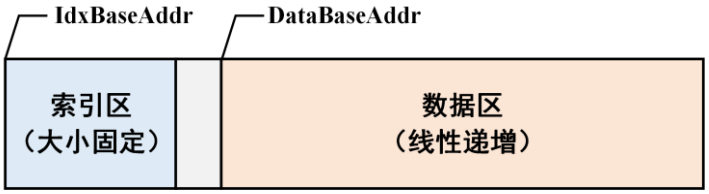



图 7. 脉冲数据在 DDR 中分区存储

在索引区中为每张图像维护一个脉冲索引表，每个索引表中存储的是每层、每个时间步、每组通道（$N_S$ 个通道一组）脉冲数据在数据区中的起始地址和数据长度（单位 Byte）。假设模型最大层数为 16，最大时间步也是 16，最大通道组数为 32，那么在初始化阶段，每张索引表如表 4 所示：

表 4. 脉冲索引表

| (*l*, *t*, *c_group*) | BaseAddr_*t* (32 bits) | Length (32 bits) |
|---|---|---|
| (0, 0, 0) | 0x0001_0000 | 1K |
| (0, 0, 1) | 0x0001_0400 | 2K |
| (0, 0, 2) | 0x0001_0C00 | 1K |
| ... | ... | ... |
| (0, 0, 31) | 0x0001_F000 | 512 |
| (0, 1, 0) | 0x0001_F200 | 1K |
| (0, 1, 1) | 0x0001_F600 | 2K |
| ... | ... | ... |
| (0, 1, 31) | 0x0002_0000 | 512 |
| … | … | … |
| (1, 0, 0) | 待定 | 待定 |
| (1, 0, 1) | 待定 | 待定 |
| ... | ... | ... |
| (1, 0, 31) | 待定 | 待定 |

观察表 4，发现在初始化阶段只写了 $l$=0 层的信息（数据是随便写的），这是因为这一层就是输入层，也就是图像层，后续的层信息片外是无法预知的，需要加速器进行补充。每张索引表的大小为 16×16×32×64b = 512 Kb =64KB。在初始化数据阶段，假设需要处理 1024 张图像，那么上位机需要把所有图像的索引表和每个时间步的脉冲数据在 DDR 中加载好。在数据区写数据时按照通道顺序依次写入 AER 即可，不同通道数据间用 CEnd 标记隔开，和片内保持一致。在索引表中，每个时间步的数据长度 Length 可以任意，但 BaseAddr_*t* 最好按照 128-bit/256-bit 对齐，以方便取址。在上位机完成初始化后，需要把两个指针写入加速器内置的参数寄存器，第一个是索引区的起始地址 IdxBaseAddr，这样加速器

才知道从哪个位置开始读索引表；第二个是全局写指针 GlbWrPtr，这个指针指示了从数据区的哪个位置继续写数据，是动态递增的。假设 DataBaseAddr = 0x0001_0000，初始化时上位机已经往数据区写了 1024 张图像共 1MB 数据，那么初始 GlbWrPtr 就是 0x0011_0000。加速器中还配置了 4 个计数器，分别是图像、层、时间步和通道计数器，由于索引表的大小是固定的（64KB），因此只需要通过图像计数器就可计算出任意图像索引表的起始地址；然后通过层和时间步计数器来访问索引表，就可以知道某层某时间步的脉冲数据在 DDR 中的起始地址和长度。图 8 进一步展示了索引和数据在 DDR 中的存储方式，以方便理解。

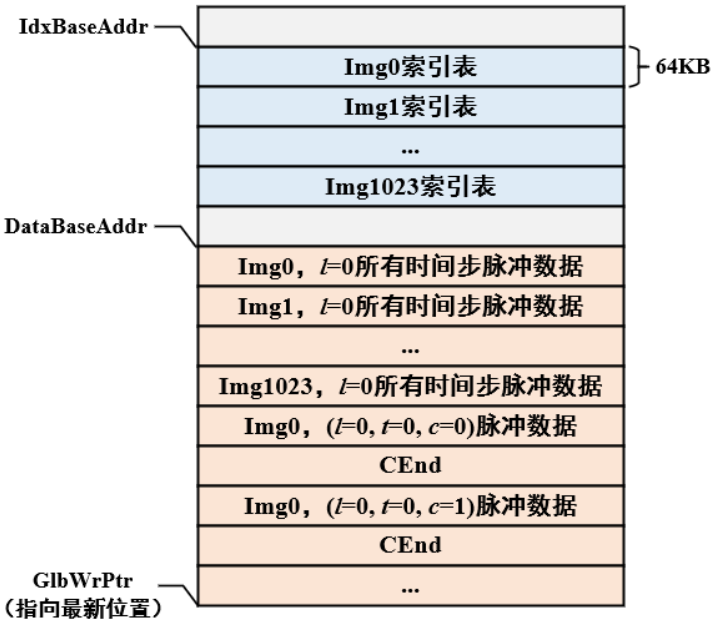



图 8. DDR 中的脉冲数据存储方式

目前为止，在 DDR 这一侧的准备工作已经完成，下面讨论加速器的行为。首先，片内脉冲缓存同样采用乒乓存储，在大模型（L）版本中对单块（乒或乓）使用了 648Kb（18 块 BRAM）来缓存，由于在小模型（S）版本中，无论 AER 的比特数还是各通道脉冲数量都大幅减小了，因此单块缓存大小可以设置为 144Kb（4 块 BRAM），实际实现时可以把这里参数化以方便调整，无论缓存多大，控制逻辑都是不变的。在加速器开始处理时，首先从 DDR 中读取第 1 张图的索引表并缓存至片上（可以一次只读索引表中的一层，只需要 1 块 BRAM），然后从表中读取第 1 个时间步（$l$=0, $t$=0）的脉冲数据信息，即在内存中的起始地址和长度。这里需要分情况处理，假设数据长度小于单个缓存，则可以全部读到乒缓存中；如果大于单个缓存，则需要进行简单计算后分多次读取。乒缓存中有 4 个 Bank，需采用路由器把内存中的数据分发到每个 Bank 中，注意 CEnd 信号要复制 4 份。当乒缓存准备就绪后，加速器开始计算，同时继续读取剩余数据到乓缓存。这里需要再回顾一下加速器的行为，假设有 16 个空间引擎并行计算 16 个输

出通道，当这 16 个输出通道的所有时间步所有输入通道脉冲计算完后，阈值引擎开始工作，它需要遍历每个输出通道的膜电位来发射脉冲。需注意，阈值引擎只能有 1 个，而不能 16 个并行工作，这是因为脉冲必须按照通道顺序存储（通道内顺序无所谓），如果有 16 个阈值引擎同时发射 16 个通道的脉冲，那 CEnd 信号就失效了，而片内控制逻辑都是依赖 CEnd 信号的。阈值引擎发射的脉冲需要缓存在一个 FIFO 中，然后使用 Burst Write 向 DDR 写数据。写入时最好采用长突发，比如凑够 1KB、2KB 数据再写，以提升传输效率。单个 BRAM 大小为 36Kb=4.5KB，这个 FIFO 只需要一个 BRAM 实现就够了。在向内存中写入数据时需要不断更新 GlbWrPtr，以向内存中连续写入脉冲，所以说数据区是线性递增的。同时片上还需要更新每个时间步的脉冲计数器，因为片上处理逻辑是每 16 个输出通道遍历所有时间步，而不是逐时间步处理的，因此需要 16 个计数器来统计每个时间步的脉冲数量，用于更新片上缓存的索引表。这样在处理第二层的时候，第一层索引表已更新完毕，GlbWrPtr 也在最新位置，因此可以持续不断地处理下去。当最后一层计算完后需要将索引表写回 DDR，然后重复上述流程，开始处理第二张图。下面总结一下这种内存管理机制的特点：

- 充分适配脉冲稀疏性。脉冲存储的难点在于，由于其稀疏性和不可预测性，难以像传统做法那样预先在 DDR 中分配地址、或缓存固定通道的脉冲，这样很容易导致内存碎片化严重、缓存溢出或利用率低下。而本文的设计动态适配稀疏脉冲，始终可以保持高内存和缓存利用率；
- 最大化 DDR 带宽利用率。首先避免了随机读写，本文通过独立存储元数据（索引区）和脉冲数据（数据区），把脉冲数据像记录磁带一样紧凑、顺序地在数据区追加写入，并且都是凑齐一个 Burst 再写，这对 DDR 控制器来说是理想的模式，可以接近 DDR 的最大带宽；
- 片上控制逻辑简单且资源开销低。本设计无需采用复杂的链表管理，核心逻辑仅依赖于索引表和全局写指针 GlbWrPtr，且索引表和脉冲写缓存 FIFO 等资源开销都极小；
- 这种设计的缺陷是需要重复访问 DDR，这是因为每一组输出通道都要使用相同的输入脉冲，但又没办法一口气把所有输出通道处理完，因此每次只能重复访问。但这属于架构的特点，通过这种数据流设计，无需片外缓存膜电位，权重也可以一定程度复用，与片外的交互降到了最低，但代价是脉冲数据的重复访问。这就是芯片设计中需要做出的权衡。

上面所有努力的核心目标就是增大数据传输带宽，并掩盖 DDR 读写延迟。围绕这两个目标，需要说明一下在实际实现过程中的注意事项：

- 尽量用移位代替乘除法，避免控制逻辑成为时序瓶颈，地址和数据能对

齐到 2 的整数次幂就尽量对齐，例如把 BaseAddr_*t* 按照 128-bit/256-bit 对齐。该原则在整个加速器控制逻辑设计中均适用，即使多耗费一些存储资源也没关系，因为目标是高性能加速，时钟频率至少需要达到 300MHz，因此这些实现这些控制逻辑时要加倍小心；

- 如果片上处理太快，导致计算时间小于读数据时间，那么需要增大片上脉冲乒乓缓存才能掩盖 DDR 延迟，这个可以最后再来分析；
- DDR 读写必须使用 Burst Read 和 Burst Write，否则数据传输效率太低。具体 Burst 大小可以根据实际情况决定；
- DDR 和处理器之间不使用 DMA，而是使用 AXI DataMover IP。这是因为 DMA 需要加速器充当 AXI Master 去配置源地址、目的地址、各种寄存器等，太复杂；而使用 AXI DataMover 时控制器只需要组装一个长指令（包含地址和数据长度等），通过 AXI-Stream 接口流式发送给 AXI DataMover，在片上无需复杂的总线交互。

# 6 数据调度器

数据调度器负责所有缓存的地址计算和数据读写，把读取的数据按照正确方式组织后发送给核心计算引擎，然后接收膜电位和脉冲运算结果，进行一定处理后再写入缓存器，整体功能较为复杂，下面对各部分功能分别进行说明：

## 6.1 膜电位取址和重排

由于是 16 个神经元并行处理，为了最大化吞吐率，膜电位缓存被划分为 16 个 Bank。同时，为了让每个 AER Tile 对应的膜电位都分布在不同的 Bank 里，需要采用交错存储[1]。由于原文讲的不是很清楚，这里重新整理一下文献[1]中的处理方式（事件驱动，3×3 膜电位并行处理，膜电位缓存被划分为 9 个 Bank）：

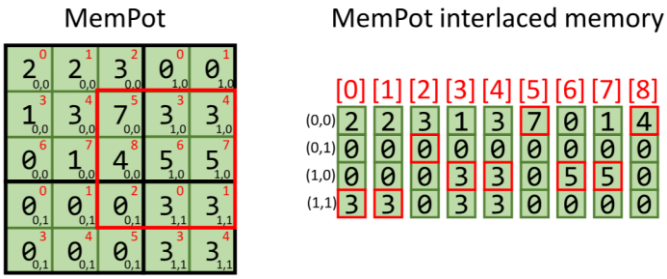



图 9. 膜电位交错存储[1]

如图 9 所示，MemPot 的每个小格子中右上角的红色数字表示 Mem Bank 的编号，一共 9 个 Bank，所以是 0-8。中间的数字代表膜电位值，右下角的数字表示每个 3×3 块在空间中的坐标。所以这种存储机制可以直白的概括为“一块一块存”，首先把左上角坐标为(0, 0)的 3×3 块存储在 9 个 Bank 的物理地址 0，然后把这个 3×3 块向下滑动（步长为 3，不重复存储）到坐标(0, 1)，存储到 9 个 Bank

的物理地址 1。此例中特征图大小为 5×5，因此向下滑到底之后跳转到右上坐标位置(1, 0)，然后存储到 9 个 Bank 的物理地址 2。最后再滑到坐标(1, 1)，存储在 9 个 Bank 的物理地址 2。可以发现，每个 3×3 块中的元素在 9 个 Mem Bank 中的地址都是一样的，因此只需要知道 3×3 块的坐标($i_{mem}$, $j_{mem}$)，就可以计算出其中元素在 Mem 中的物理地址：

$$\text{MemAddr} = i_{mem} \times N_{\text{blocks per column}} + j_{mem} \tag{1}$$

接下来讨论如何根据 AER 地址来进行膜电位取址。首先，如图 10 所示，脉冲也是交错存储在 9 个独立的 Mem Bank 中的，每个 Bank 的编号与膜电位对应，例如膜电位 Bank0 发射的脉冲就存储在脉冲 Bank0 中。需要注意的是，Bank 中存储的 AER 地址并不是细粒度地址($x$, $y$)，而是 3×3 块的粗粒度坐标($i_{in}$, $j_{in}$)。在 3×3 块中，每个 AER 的偏移量是隐含在 Bank 的编号中的。通过这种隐式偏移，实际存储 AER 所需的比特数有所下降。

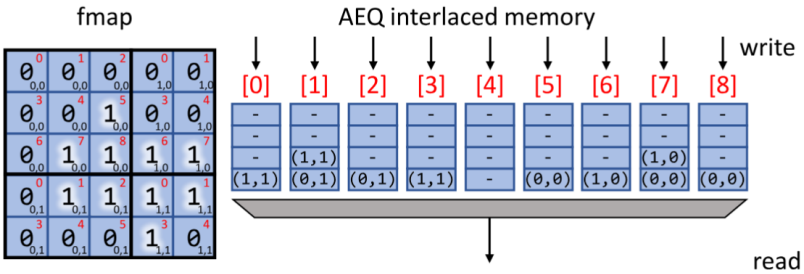



图 10. 脉冲交错存储[1]

因此只需要知道每个 AER 的粗粒度坐标($i_{in}$,$j_{in}$)和偏移量 $s_{in}$（即 Bank 编号），就可以计算出每个膜电位 3×3 块坐标($i_{mem}$,$j_{mem}$)，如公式（2）所示（假设输入都是 padding 过的，输入输出大小相等）：

$$i_{mem} = \begin{cases} i_{in} + 1, & \text{if } s_{in} \in \{2, 5, 8\} \\ i_{in}, & \text{otherwise} \end{cases}, \text{for } s_{mem} = 0$$
$$j_{mem} = \begin{cases} j_{in} + 1, & \text{if } s_{in} \in \{6, 7, 8\} \\ j_{in}, & \text{otherwise} \end{cases}, \text{for } s_{mem} = 0. \tag{2}$$

公式（2）展示了膜电位 Bank0（即 $s_{mem}$=0）块坐标（$i_{mem}$,$j_{mem}$）的计算方法。这种形式是考虑了跨界的情况。对于 Bank0 来说，当 $s_{in}$ ∈{2, 5, 8}时，该卷积窗口会跨界到 Bank2（$i_{mem}$=1,$j_{mem}$=0）去，因此 $i_{mem}$=$i_{in}$+1；而当 $s_{in}$∈{6, 7, 8}时，该卷积窗口会跨界到 Bank1（$i_{mem}$=0, $j_{mem}$=1）去，因此 $j_{mem}$=$j_{in}$+1，其它情况对于 Bank0 来说都不会跨界。下面补充了所有 Bank 的块坐标计算表，请对照表格确认理解无误。可以看出，需要为每个 Bank 配置一套独立的地址计算逻辑。

表 1. 各 Bank 块坐标计算表

| Bank 编号 | $i_{mem}$ 计算 | $j_{mem}$ 计算 |
|---|---|---|
| 0 | 当 $s_{in}$∈{2, 5, 8}, $i_{in}$ + 1 | 当 $s_{in}$∈{6, 7, 8}, $j_{in}$ + 1 |

| | | |
|---|---|---|
| 1 | 始终等于 $i_{in}$ | 当 $s_{in} \in \{6, 7, 8\}$, $j_{in} + 1$ |
| 2 | 当 $s_{in} \in \{0, 3, 6\}$, $i_{in} - 1$ | 当 $s_{in} \in \{6, 7, 8\}$, $j_{in} + 1$ |
| 3 | 当 $s_{in} \in \{2, 5, 8\}$, $i_{in} + 1$ | 始终等于 $j_{in}$ |
| 4 | 始终等于 $i_{in}$ | 始终等于 $j_{in}$ |
| 5 | 当 $s_{in} \in \{0, 3, 6\}$, $i_{in} - 1$ | 始终等于 $j_{in}$ |
| 6 | 当 $s_{in} \in \{2, 5, 8\}$, $i_{in} + 1$ | 当 $s_{in} \in \{0, 1, 2\}$, $j_{in} - 1$ |
| 7 | 始终等于 $i_{in}$ | 当 $s_{in} \in \{0, 1, 2\}$, $j_{in} - 1$ |
| 8 | 当 $s_{in} \in \{0, 3, 6\}$, $i_{in} - 1$ | 当 $s_{in} \in \{0, 1, 2\}$, $j_{in} - 1$ |

接下来是数据重排问题，由于 PE 阵列的空间位置固定，而膜电位在内存中是交错存储的，因此需要重排数据。但是膜电位精度较高，而权重精度较低，因此文献[1]选择重排权重，为每个 PE 配置了一个 9-to-1 MUX，把 $s_{in}$ 作为选通信号进行选择。以 PE0 为例，来看看是如何选择的：

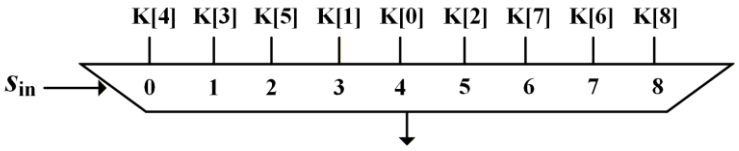



图 11. 针对 PE0 的权重重排

方法就是看 PE0 在卷积核中的位置。例如，当 $s_{in}$=0 时，以该点为中心画一个 3×3 区域，PE0 在区域的正中，对应权重 K[4]；当 $s_{in}$=1 时，以该点为中心画一个 3×3 区域，PE0 在区域的左中，对应权重 K[3]；当 $s_{in}$=8 时，以该点为中心画一个 3×3 区域，PE0 在区域的右下（需要跨界），对应权重 K[8]。由于有 9 个 PE，因此需要配置九套 9-to-1 MUX。

搞清楚原文的设计方法后，回到我们的设计，每次读出一个 2×2 AER Tile，同样需要 9 个权重，但是需要并行处理 16 个膜电位。首先是膜电位取址的问题，假设还是采用文献[1]的方法，那么每个空间引擎都需要配置 16 套地址转换电路，每一套都包含数个加法器和比较器，过于浪费资源。因此本文采用查找表的方法，直接把所有情况都预存起来，使用时直接查表即可，避免动态计算开销。

在本设计中，阈值引擎以 stride=4 每次读出 4×4 的膜电位进行阈值判断，然后按 2×2 Tile 打包脉冲，因此脉冲缓存只需要 4 个 Bank，即查找表只需要 4 个 Entry。同样定义 AER Tile 的粗粒度（即 4×4 块）坐标($i_{in}$, $j_{in}$)，偏移量 $s_{in} \in \{0,1,2,3\}$，对应的地址偏移查找表如下：

表 2. 膜电位 Bank 地址偏移查找表

| | | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| $s_{in} = 0$ | $\Delta i_0, \Delta j_0$ | $\Delta i_1, \Delta j_1$ | $\Delta i_2, \Delta j_2$ | $\Delta i_3, \Delta j_3$ | $\Delta i_4, \Delta j_4$ | $\Delta i_5, \Delta j_5$ | $\Delta i_6, \Delta j_6$ | $\Delta i_7, \Delta j_7$ |
| | 0, 0 | 0, 0 | 0, 0 | -1, 0 | 0, 0 | 0, 0 | 0, 0 | -1, 0 |
| | $\Delta i_8, \Delta j_8$ | $\Delta i_9, \Delta j_9$ | $\Delta i_{10}, \Delta j_{10}$ | $\Delta i_{11}, \Delta j_{11}$ | $\Delta i_{12}, \Delta j_{12}$ | $\Delta i_{13}, \Delta j_{13}$ | $\Delta i_{14}, \Delta j_{14}$ | $\Delta i_{15}, \Delta j_{15}$ |
| | 0, 0 | 0, 0 | 0, 0 | -1, 0 | 0, -1 | 0, -1 | 0, -1 | -1, -1 |
| $s_{in} = 1$ | $\Delta i_0, \Delta j_0$ | $\Delta i_1, \Delta j_1$ | $\Delta i_2, \Delta j_2$ | $\Delta i_3, \Delta j_3$ | $\Delta i_4, \Delta j_4$ | $\Delta i_5, \Delta j_5$ | $\Delta i_6, \Delta j_6$ | $\Delta i_7, \Delta j_7$ |

| | | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| | +1, 0 | 0, 0 | 0, 0 | 0, 0 | +1, 0 | 0, 0 | 0, 0 | 0, 0 |
| | $\Delta i_8, \Delta j_8$ | $\Delta i_9, \Delta j_9$ | $\Delta i_{10}, \Delta j_{10}$ | $\Delta i_{11}, \Delta j_{11}$ | $\Delta i_{12}, \Delta j_{12}$ | $\Delta i_{13}, \Delta j_{13}$ | $\Delta i_{14}, \Delta j_{14}$ | $\Delta i_{15}, \Delta j_{15}$ |
| | +1, 0 | 0, 0 | 0, 0 | 0, 0 | +1, -1 | 0, -1 | 0, -1 | 0, -1 |
| $s_{in} = 2$ | $\Delta i_0, \Delta j_0$ | $\Delta i_1, \Delta j_1$ | $\Delta i_2, \Delta j_2$ | $\Delta i_3, \Delta j_3$ | $\Delta i_4, \Delta j_4$ | $\Delta i_5, \Delta j_5$ | $\Delta i_6, \Delta j_6$ | $\Delta i_7, \Delta j_7$ |
| | 0, +1 | 0, +1 | 0, +1 | -1, +1 | 0, 0 | 0, 0 | 0, 0 | -1, 0 |
| | $\Delta i_8, \Delta j_8$ | $\Delta i_9, \Delta j_9$ | $\Delta i_{10}, \Delta j_{10}$ | $\Delta i_{11}, \Delta j_{11}$ | $\Delta i_{12}, \Delta j_{12}$ | $\Delta i_{13}, \Delta j_{13}$ | $\Delta i_{14}, \Delta j_{14}$ | $\Delta i_{15}, \Delta j_{15}$ |
| | 0, 0 | 0, 0 | 0, 0 | -1, 0 | 0, 0 | 0, 0 | 0, 0 | -1, 0 |
| $s_{in} = 3$ | $\Delta i_0, \Delta j_0$ | $\Delta i_1, \Delta j_1$ | $\Delta i_2, \Delta j_2$ | $\Delta i_3, \Delta j_3$ | $\Delta i_4, \Delta j_4$ | $\Delta i_5, \Delta j_5$ | $\Delta i_6, \Delta j_6$ | $\Delta i_7, \Delta j_7$ |
| | +1, +1 | 0, +1 | 0, +1 | 0, +1 | +1, 0 | 0, 0 | 0, 0 | 0, 0 |
| | $\Delta i_8, \Delta j_8$ | $\Delta i_9, \Delta j_9$ | $\Delta i_{10}, \Delta j_{10}$ | $\Delta i_{11}, \Delta j_{11}$ | $\Delta i_{12}, \Delta j_{12}$ | $\Delta i_{13}, \Delta j_{13}$ | $\Delta i_{14}, \Delta j_{14}$ | $\Delta i_{15}, \Delta j_{15}$ |
| | +1, 0 | 0, 0 | 0, 0 | 0, 0 | +1, 0 | 0, 0 | 0, 0 | 0, 0 |

**注：正确性尚未验证，使用前请先验证**

$\Delta i_n, \Delta j_n$ 是 4×4 膜电位块中第 $n$ 个 Bank 地址计算所需的偏移，每次来一个 $s_{in}$，直接把 16 个 Bank 的 $\Delta i$ 和 $\Delta j$ 全部查询出来，每个 Bank 的物理地址坐标计算如下：

$$\text{MemAddr}[n] = (i_{in} + \Delta i_n[s_{in}]) + (j_{in} + \Delta j_n[s_{in}]) \times N_{\text{blocks per row}} \tag{3}$$

需注意，公式（1）是列优先存储，公式（3）调整为行优先，这更符合图像处理习惯。$\Delta i$ 和 $\Delta j$ 分别需要 2-bit 表示，因此查找表的大小为 64b×4，用 LUTRAM 实现即可。在计算公式（3）时，为了避免乘法器，需要把硬件视角的特征图大小扩展为 2 的整数次幂。以 Spiking VGG11 第一层 224×224 为例，软件计算不变，硬件存储时看作 256×256，以少量冗余存储空间为代价，将乘 $N_{\text{blocks per row}}$ 变为移位计算。

接下来是膜电位的重排问题，由于文献[1]采用了同构的 PE，因此无论重排膜电位还是权重都可以。但在本文图 5 中，如果把每个神经元看作一个 PE，那么每个 PE 的结构和算力都是不同的，相当于一种异构设计。因此只能把角点膜电位分配给角点 PE，中心膜电位分配给中心 PE，必须要重排膜电位。如果此时仍然采用文献[1]的设计，那么需要 16 个 16-to-1 MUX，在 FPGA 上的逻辑资源开销不算大，但是会给布线带来很大压力，很有可能会成为时序瓶颈。

16 个 16-to-1 MUX 是基于一个假设，那就是每个神经元都有可能访问任意的 16 个膜电位 Bank。然而，仔细分析我们的设计，由于采用了 2×2 AER Tile，相当于滑动的 Stride=2，那么对于每个神经元，实际上只能访问其中的 4 个 Bank。如下图，对于神经元 0 而言，由于滑动 Stride=2，其永远只能访问 Bank5, 7, 13, 15。

| | | | |
|---|---|---|---|
| 15 | 12 | 13 | 14 |
| 3 | 0 | 1 | 2 |
| 7 | 4 | 5 | 6 |
| 11 | 8 | 9 | 10 |

基于上述推论，16 个 16-to-1 MUX 被简化为 16 个 4-to-1 MUX，硬件资源和布线压力都大幅降低了。实际实现时这些 MUX 直接硬连线即可，类似图 11。下表列

出了这 16 个 MUX 的选通情况：

表 3. 膜电位重排各 MUX 连线表

| | MUX0 | MUX1 | MUX2 | MUX3 | MUX4 | MUX5 | MUX6 | MUX7 |
|---|---|---|---|---|---|---|---|---|
| $s_{in}=0$ | 15 | 12 | 13 | 14 | 3 | 0 | 1 | 2 |
| $s_{in}=1$ | 13 | 14 | 15 | 12 | 1 | 2 | 3 | 0 |
| $s_{in}=2$ | 7 | 4 | 5 | 6 | 11 | 8 | 9 | 10 |
| $s_{in}=3$ | 5 | 6 | 7 | 4 | 9 | 10 | 11 | 8 |
| | | | | | | | | |
| | MUX8 | MUX9 | MUX10 | MUX11 | MUX12 | MUX13 | MUX14 | MUX15 |
| $s_{in}=0$ | 7 | 4 | 5 | 6 | 11 | 8 | 9 | 10 |
| $s_{in}=1$ | 5 | 6 | 7 | 4 | 9 | 10 | 11 | 8 |
| $s_{in}=2$ | 15 | 12 | 13 | 14 | 3 | 0 | 1 | 2 |
| $s_{in}=3$ | 13 | 14 | 15 | 12 | 1 | 2 | 3 | 0 |

**注：正确性尚未验证，使用前请先验证**

由于 FPGA 上的最小单元 LUT 是 6 输入的，膜电位位宽为 16bit，因此只需要 256 个 LUT6 即可实现上述膜电位重排逻辑。实际硬件实现时请检查此处的资源消耗是否正确。

重排后的膜电位得到处理后，在写入膜电位缓存前需要进行逆重排（空间逆映射），以免发生数据错乱，映射方式如下表。逆重排所消耗的资源与重排一致。

表 4. 膜电位**逆重排**各 MUX 连线表

| | MUX0 | MUX1 | MUX2 | MUX3 | MUX4 | MUX5 | MUX6 | MUX7 |
|---|---|---|---|---|---|---|---|---|
| $s_{in}=0$ | 5 | 6 | 7 | 4 | 9 | 10 | 11 | 8 |
| $s_{in}=1$ | 7 | 4 | 5 | 6 | 11 | 8 | 9 | 10 |
| $s_{in}=2$ | 13 | 14 | 15 | 12 | 1 | 2 | 3 | 0 |
| $s_{in}=3$ | 15 | 12 | 13 | 14 | 3 | 0 | 1 | 2 |
| | | | | | | | | |
| | MUX8 | MUX9 | MUX10 | MUX11 | MUX12 | MUX13 | MUX14 | MUX15 |
| $s_{in}=0$ | 13 | 14 | 15 | 12 | 1 | 2 | 3 | 0 |
| $s_{in}=1$ | 15 | 12 | 13 | 14 | 3 | 0 | 1 | 2 |
| $s_{in}=2$ | 5 | 6 | 7 | 4 | 9 | 10 | 11 | 8 |
| $s_{in}=3$ | 7 | 4 | 5 | 6 | 11 | 8 | 9 | 10 |

## 6.2 AER 乱序与流水线冒险

由 5.3 小节所述，将 DDR 中的脉冲数据缓存至脉冲缓存的 4 个 Bank 中时，需要通过一个路由器来进行分发。路由器可以按照空间交错的方式对 AER Tile 进行路由，即在 4×4 块中偏移值相同的 AER Tile 路由至同一个 Bank，4 个 Bank

分别缓存不同偏移值的 AER Tile。通过这种路由方式，每个 Bank 中的脉冲确保不会相邻，由于加速器是逐 Bank 处理的，脉冲相邻的情况只可能出现在切换 Bank 时。

注意在小模型（S）版本中，网络深层特征图尺寸会缩小为 4×4 和 2×2，此时无论采用什么乱序机制投影的膜电位都 100%相邻。首先，无论是 100%相邻还是多大概率相邻，计分板（Scoreboard）单元都是必须的。例如，假设空间引擎的流水线深度为 4，那么就在计分板中使用 4 组寄存器记录当前正在流水线中处理的 4 个 AER Tile 的地址，当下一个 AER Tile 到来时，首先与这 4 个地址进行比较，如果有相邻的情况则直接暂停流水线，等待冲突 Tile 对应的膜电位都写回完成再继续处理，以确保读出的是最新的膜电位数据。在此基础上，考虑到小模型（S）的特征图小，导致膜电位相邻概率高（特别是 4×4 和 2×2 概率为 100%），一共有三种解决方案：

1）空间引擎 DSP 采用混合模式。对于 4×4 和 2×2 的层，由于神经元数量不超过 16 个，因此可以改变空间引擎 DSP 的模式，将其配置为和向量引擎一样的输出固定模式，每个 DSP 负责两个固定位置神经元的膜电位计算，并利用 DSP 自带的寄存器连续累加，直到该输出通道处理完才把膜电位写回缓存。优势：彻底消除了流水线冒险；劣势：数据调度器和空间映射单元必须额外维护一套完全不同的寻址与路由逻辑，增加了布线复杂度，导致这里很有可能成为时序瓶颈；

2）数据旁路（Bypassing）。建立数据反馈旁路，当计分板检测到新 AER Tile 与流水线中某个尚未写回的 Tile 相邻时，不让流水线彻底停顿，而是增加旁路选择 MUX，直接从后级流水线（如最后一级 DSP 的输出端）将刚刚算出的最新膜电位结果直接拿过来用，而不是先写进缓存再读取。优势：无需改变原有的数据流和内存寻址模式，且可以让流水线少停顿 2 个周期（节约了膜电位的写和读周期）；劣势：神经元累加阵列的整体流水线深度可能较深，无法完全避免停顿的开销，且旁路这条路径有可能成为瓶颈；

3）什么都不做。流水线停顿就让它停顿，不增加额外的电路。优势：控制最为简单，无额外开销；劣势：对于 4×4 和 2×2 这些层，流水线中有大量气泡，几乎处于半瘫痪状态。

对于硬件的第一个版本，直接采用方案 3）即可，4×4 和 2×2 这些层尽管通道数非常多，但特征图尺寸小，且越深层越稀疏，它们的整体计算量占比可能并不是很大，因此即使流水线停顿，从整体看也不一定会成为瓶颈，因此可以先采用方案 3）做，做完再来分析，如果确实是性能瓶颈再想办法。

# 参考文献

[1] J. Sommer, M. A. Özkan, O. Keszocze and J. Teich, "Efficient Hardware Acceleration of Sparsely Active Convolutional Spiking Neural Networks," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 41, no. 11, pp. 3767-3778, Nov. 2022.

[2] J. Li, G. Shen, D. Zhao, Q. Zhang and Y. Zeng, "FireFly: A High-Throughput Hardware Accelerator for Spiking Neural Networks With Efficient DSP and Memory Optimization," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 31, no. 8, pp. 1178-1191, Aug. 2023.